



Arrays

(CS141, slides by Jaime Spacco, edits by Vera Kazakova)

What is an array?

An array is a **data structure** containing an ordered collection of some *datatype*

For example, an array of int is an ordered collection of ints.

Consider a list of int values 1, 2, 4, 8, 16. Five values, of type int, in that specific order.

Can you think of some cases where this is useful?

What is an array?

An array is a **data structure** containing an ordered collection of some *datatype*

For example, an array of int is an ordered collection of ints.

Consider a list of int values 1, 2, 4, 8, 16. Five values, of type int, in that specific order.

Can you think of some cases where this is useful?

- names of everyone in the class?
(an array of ~75 String variables)
- how many days each month has?
(an array of 12 int variables)
- rainy days this week?
(an array of 7 booleans)

Let's look at a sample int array

1	2	4	8	16
[0]	[1]	[2]	[3]	[4]

array elements

array indices



`int[ ] a` is an integer array of length 5, so its indices are 0, 1, 2, 3, 4

Let's look at a sample int array

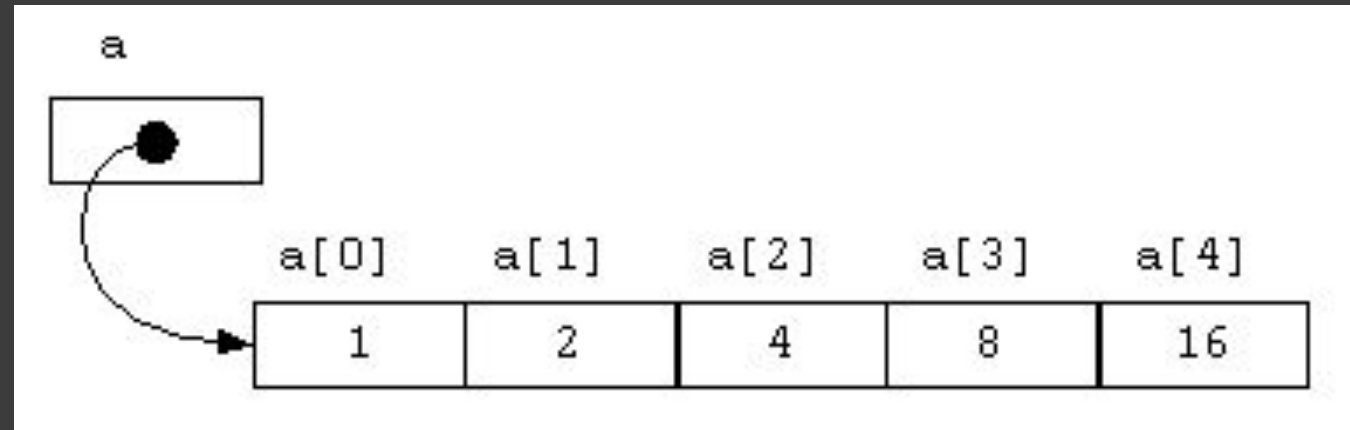
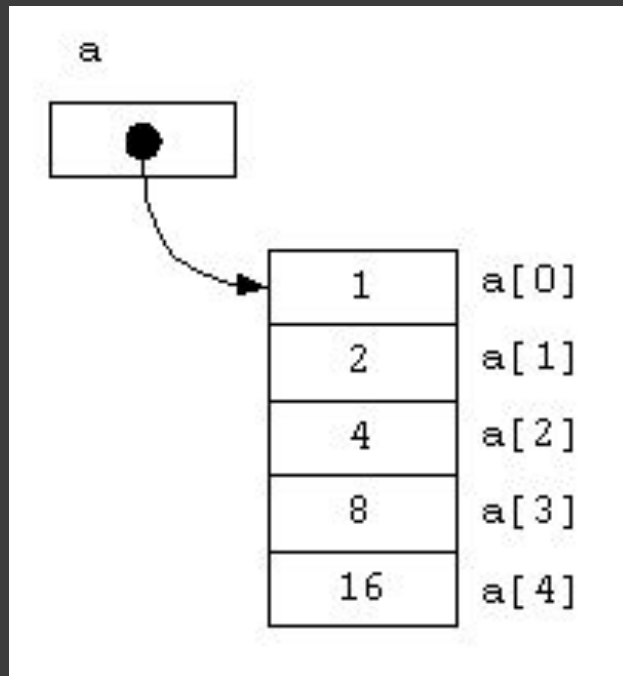
1	2	4	8	16
[0]	[1]	[2]	[3]	[4]

array elements

array indices



`int[ ] a` is an integer array of length 5, so its indices are 0, 1, 2, 3, 4



You may see arrays depicted as a row or a column;
What matters is that they are stored consecutively somewhere

Creating an array of int

```
int[] theArray = new int[6];
```

Creating an array of int

variable name

we use new like we do for constructors

```
int[] theArray = new int[6];
```

brackets after type indicate array

put the size inside brackets with the type

Creating an array of int

variable name

we use new like we do for constructors

```
int[] theArray = new int[6];
```

brackets after type indicate array

put the size inside brackets with the type

We now have a new variable called theArray.

It is an array with room for 6 integers at indices 0 to 5:

theArray[0], theArray[1], theArray[2], theArray[3], theArray[4], theArray[5]

Putting data into an array

```
int[] arr = new int[6];  
arr[0] = 10;
```

Putting data into an array

```
int[] arr = new int[6];  
arr[0] = 10;
```

We use square brackets to access a specific spot in the array.

This is called *indexing into* the array.

Arrays hold different values at each location.

Quick array initializer

If you already know what elements you want in an array, you can do this:

```
int[] arr = new int[] {1,2,3,4};
```

Equivalent to this:

```
int[] arr=new int[4];  
arr[0]=1;  
arr[1]=2;  
arr[2]=3;  
arr[3]=4;
```

Getting data out of an array

```
int[] arr = new int[6];  
arr[0] = 10;
```

```
System.out.println(arr[0]); //this prints 10
```

The length of an array

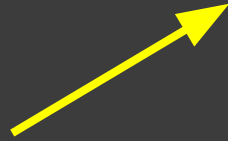
```
int[] arr = new int[10];
```

```
println("length is "+arr.length);
```

The length of an array

```
int[] arr = new int[10];
```

```
println("length is "+arr.length);
```



Note that the length is *not* a method (NO PARENS AFTER IT; IT'S NOT A CALL).
It's an int field (a variable) inside the array that's called length!

What is `int[]`, primitive or reference?

A: primitive type

B: reference type

What is `int[]`, primitive or reference?

A: primitive type

B: reference type

All arrays are reference types, even if they hold primitive type data elements.

This is what an array of primitive datatype looks like in memory:

theArray



theArray is the name of the array variable

```
int[ ] theArray = new int [4];
```

It references an array of size 4,
with default value of 0 for each of its elements.

0	0	0	0
[0]	[1]	[2]	[3]

This is what an array of reference datatype looks like in memory:

theArray



theArray is the name of the array variable

```
String[ ] theArray = new String[4];
```

It references an array of size 4,
with default value of null for each of its elements.

null	null	null	null
[0]	[1]	[2]	[3]

This is what an array of reference datatype looks like in memory:

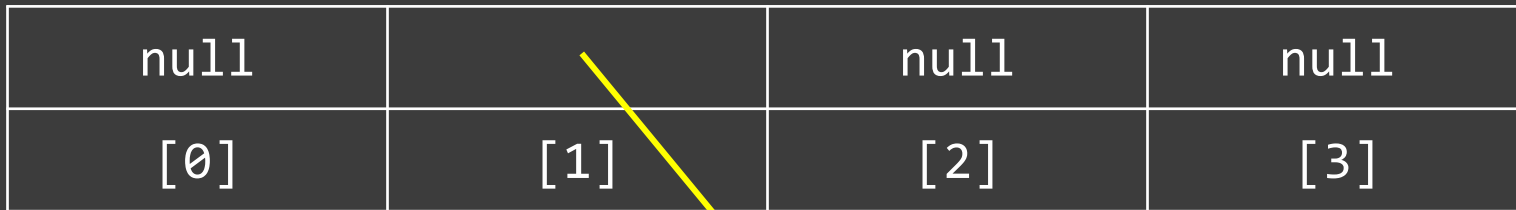
theArray

theArray is the name of the array variable



```
String[ ] theArray = new String[4];
```

```
theArray[1] = "I am Groot";
```



null		null	null
[0]	[1]	[2]	[3]

"I am Groot"

What does this print?

```
int[] arr = new int[10];  
System.out.println(arr[0]);
```

- A: 0
- B: 10
- C: -1
- D: it doesn't compile
- E: it generates an exception

What does this print?

```
int[] arr = new int[10];  
System.out.println(arr[0]);
```

- A: 0
- B: 10
- C: -1
- D: it doesn't compile
- E: it generates an exception

arrays are initialized with default values for the type

Default values:
int is 0,
boolean is false,
double is 0.0

What value does s have?

```
String[] sarr = new String[10];  
String s = sarr[0];
```

- A: "" (empty String)
- B: null
- C: 0
- D: doesn't compile
- E: generates runtime exception

What value does s have?

```
String[] sarr = new String[10];  
String s = sarr[0];
```

- A: "" (empty String)
- B: null**
- C: 0
- D: doesn't compile
- E: generates runtime exception

Arrays of reference datatypes (classes) use **null** as the default value, meaning *no reference*.

What value does len have?

```
String[] sarr = new String[10];  
String s = sarr[0];  
int len = s.length();
```

- A: "" (empty String)
- B: null
- C: 10
- D: doesn't compile
- E: generates a runtime exception

What value does len have?

```
String[] sarr = new String[10];  
String s = sarr[0];  
int len = s.length();
```

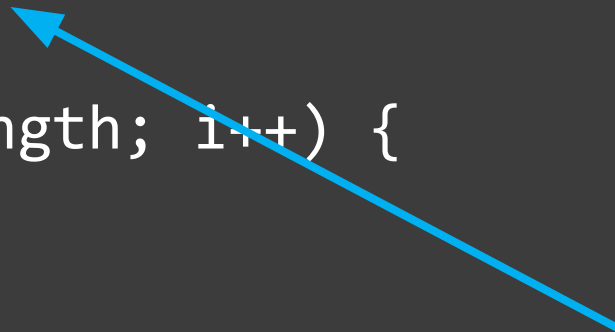
- A: "" (empty String)
- B: null
- C: 10
- D: doesn't compile
- E: generates a runtime exception

This is a classic NullPointerException!

Whenever you try to invoke a method on null, you are guaranteed a NPE!
Instance methods need an instance. null is nothing so we have no instance!

Arrays as method parameters

```
public static int func(int[] nums) {  
    int count=0;  
    for (int i=0; i<nums.length; i++) {  
        if (nums[i] > 0) {  
            count++;  
        }  
    }  
    return count;  
}
```



If you want to pass an array
as a parameter,
just remember the square
brackets in addition to the
datatype

What does this function do?

```
public static int func(int[] nums) {  
    int count=0;  
    for (int i=0; i<nums.length; i++) {  
        if (nums[i] > 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

- A: remove the positive values
- B: count the positive values
- C: count the odd values
- D: remove the even values
- E: sum the positive values

What does this function do?

```
public static int func(int[] nums) {  
    int count=0;  
    for (int i=0; i<nums.length; i++) {  
        if (nums[i] > 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

A: remove the positive values

B: count the positive values

C: count the odd values

D: remove the even values

E: sum the positive values

DONE FOR NOW

If there's time, Arrays-1 /
Arrays-2 on codingbat